

CodeBLEU分数参考：

codebleu_results						
Problem Name	Comparison Type	CodeBLEU Score	ngram_match	weighted_ngram_match	syntax_match	dataflow_match
minimum-moves-to-clean-the-classroom	Correct vs LLM	0.5404208068067520	0.5145464443030730		0.8	0.3227848101265820
minimum-moves-to-clean-the-classroom	Correct vs Faulty	0.591975448076922	0.44255671183835100		0.8415094339622640	0.6075949367088610
shortest-matching-substring	Correct vs LLM	0.7182991887951550	0.6128167076070940		0.8571428571428570	0.7947019867549670
shortest-matching-substring	Correct vs Faulty	0.9779611091429340	0.9633525422525080		0.9705882352941180	1.0
minimum-swaps-to-sort-by-digit-sum	Correct vs LLM	0.6506776592780650	0.6379248519746330		0.8516129032258060	0.2912621359223300
minimum-swaps-to-sort-by-digit-sum	Correct vs Faulty	0.9553757131356760	0.9649029643253930		0.9096774193548390	0.9805825242718450
count-covered-buildings	Correct vs LLM	0.6463394900528070	0.4299902423742700		0.7777777777777780	0.7831325301204820
count-covered-buildings	Correct vs Faulty	0.8224436363694920	0.7902290005133990		0.8296296296296300	0.7831325301204820

Judge by AI分数参考：

semantic_comparison			
Problem_Name	Comparison_Type	Semantic_Score	Reasoning
count-covered-buildings	Correct vs LLM	1.0	核心逻辑完全一致，同样使用四个极值数组做两遍扫描，唯一区别是 Math.min/max 与 if 比较的写法，语义等价。
count-covered-buildings	Correct vs Faulty	0.55	将独立的 min/max 更新改成 if-else，导致边界情形下只会执行一个分支，极值维护不完整，后续计数会出错。
minimum-moves-to-clean-the-classroom	Correct vs LLM	0.97	BFS 分层、状态四元组、能量与掩码转移规则完全一致，仅队列容器用 LinkedList 代替 ArrayList，语义等价。
minimum-moves-to-clean-the-classroom	Correct vs Faulty	0.9	整体 BFS 与状态设计一致，但把能量耗尽判断从出队时改为入队时，且邻居越界判断顺序不同，边界处理不完全对齐。
minimum-swaps-to-sort-by-digit-sum	Correct vs LLM	1.0	排序规则、映射表构建、置换环计数逻辑完全一致，仅 Integer.compare 与减法比较的写法差异，语义等价。
minimum-swaps-to-sort-by-digit-sum	Correct vs Faulty	0.3	映射表 d 错误地以数位和为键而非原值，且环遍历时用 f(nums[j]) 取代 nums[j]，键值对应关系被破坏，置换环逻辑失效。
shortest-matching-substring	Correct vs LLM	1.0	三段模式的 KMP/LPS 预处理与三指针滑动窗口完全一致，核心算法与边界条件均等价。
shortest-matching-substring	Correct vs Faulty	0.6	第三个指针 k 的下界由 j+nc 改为 j+nc-1，允许三段区间重叠一个字符，破坏互不重叠约束，可能返回更短的错误结果。

从CodeBLEU的相似性分数可以看出，该评判标准更侧重于代码在形式结构上的相似性，而相对于代码的逻辑性来说则评分较低。（测试样例中错误代码与正确代码的相似度都显著高于LLM生成的代码与正确代码之间的相似度）错误代码对比起LLM生成的代码，在语义匹配这个参数上明显更高。综合这些指标进行分析可以得出仅靠静态特征匹配可能无法真实反映代码的语义一致性。

5305 Assignment1 Stage2 AI comparison



你是一个具备本地代码库读取与分析能力的资深 AI Agent。你的任务是遍历指定的本地文件夹，读取 Java 代码文件，并评估不同来源的代码与标准正确代码之间的“语义相似度”。

【本地环境与工作目录】

请直接访问并读取以下位于我的 Mac 上的绝对路径：

- 正确代码库 (Correct Code): /Users/soupk/Desktop/5305 Task1Stage2/ARIN5305-26F-main/resource/correctJavaPrograms (注意：包含子文件夹嵌套结构)
- 错误代码库 (Faulty Code): /Users/soupk/Desktop/5305 Task1Stage2/ARIN5305-26F-main/resource/faultyJavaProgramsToCompare (注意：包含子文件夹嵌套结构)
- LLM 生成代码 (LLM Code): /Users/soupk/Desktop/5305 Task1Stage2/LLM codes (注意：所有文件平铺展示)

【任务匹配逻辑】

- 以 LLM Code 目录中的文件名为基准提取题目名称（例如从 count-covered-buildings.java 提取 count-covered-buildings）。
- 使用提取出的题目名称作为关键字，在 Correct Code 和 Faulty Code 目录中进行递归搜索，寻找对应的 .java 文件。
- 针对每一道成功匹配的題目，请分别读取文件内容，并执行两组语义比对：
 - Correct Code VS. LLM Code
 - Correct Code VS. Faulty Code

【语义相似度评估标准】

给 DeepSeek 发送消息

深度思考

智能搜索



而若让LLM直接对这些代码进行相似性分析，则是更侧重于语义之间的对比，能够准确识别出代码的正确性。按照任务具体来说，在count-covered-buildings中，在正确代码与LLM生成的代码的对比中，AI给出了1.00的高语义分数。指出了`Math.min/max`和`if`条件语句的等价替换；在正确代码与错误代码的对比中，AI给出了0.55的低分，指出了`if-else`分支中的逻辑缺陷。在minimum-moves-to-clean-the-classroom中，在正确代码与LLM生成的代码的对比中，AI给出了0.97的高分，识别出两者均采用了广度优先搜索(BFS)。差异仅在于底层数据结构(`LinkedList`与`ArrayList`)，不影响核心语义。在shortest-matching-substring中，在正确代码与LLM生成的代码的对比中，AI识别出了字符串匹配算法的差异(如提及KMP/LPS)，但在功能上保持等价；而在正确代码与错误代码的对比中，AI敏锐抓住了`j+nc`相关的关键错误，判定语义不一致。在minimum-swaps-to-sort-by-digit-sum中，在正确代码与LLM生成的代码的对比中，AI注意到了比较器的使用(如`Integer.compare`)，认可其功能一致性。

把两种评判方法放在一起看，可以发现各自的优缺点所在。就CodeBLEU而言，它具备高度自动化的特点，可以通过调用对应的库高效快捷的使用，消耗的算力较低，且对比起普通的BLEU会将语法和数据流纳入考虑，适合进行快速检测；然而该方法极易被表层文本相似度欺骗，难以识别具体的逻辑相似度。实验中的错误代码通常只改动了少数几个关键字符(以此来模拟编程时候的小失误)，而LLM生成的代码通常会重构逻辑或更换数据结构，但是CodeBLEU却无法区分这些深入到内容的不同，仅仅根据表层现象进行相似性打分。

而就AI-as-Judge而言，这种方法可以深入分析代码的逻辑性，能够揭露错误代码中的致命小错误，不会因为代码换了一种表示形式就忽略了其语义的真实逻辑所在，能够真正理解代码的底层逻辑和算法意图(如识别出BFS、KMP等高级概念)，进而准确过滤出功能等价的代码。然而使用AI进行语义分析也有不可忽视的缺点，这种方法对比起CodeBLEU而言，显著消耗更多的算力，若进行大规模的测试，运行时间也是远超前。

也许在实际应用中，可以使用CodeBLEU对大规模的数据进行初筛，然后再使用AI进一步来校验逻辑相关性。

注：以上分析均基于文档开头的两个语义相似性分析的表格。任务报告要求第三点优缺点分析所需的说明图表即为这两个表格。